

Reflection in C# / .NET

Guide to runtime type inspection and invocation: what reflection is, core APIs, attributes, plugins, performance, and when *not* to use it.

1. What reflection is

Reflection lets your code inspect and use types, members, and metadata **at runtime** — even when you did not know those names at compile time.

Normal code (compile-time knowledge):

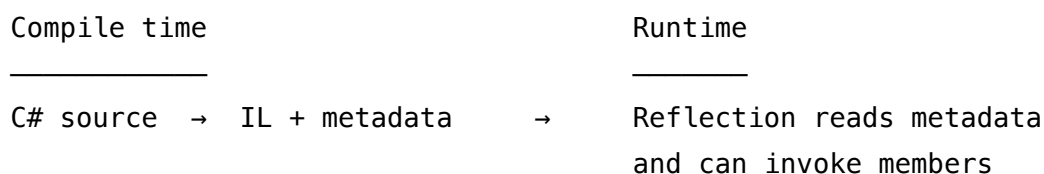
```
var user = new User();
user.Name = "Anna";
Console.WriteLine(user.Name);
```

Reflection (runtime discovery):

```
Type type = typeof(User);
object instance = Activator.CreateInstance(type)!;
PropertyInfo? nameProp = type.GetProperty("Name");
nameProp!.SetValue(instance, "Anna");
Console.WriteLine(nameProp.GetValue(instance));
```

Same idea: create object, set property, read property — but driven by **strings / Type objects** instead of hard-coded C# syntax.

Mental model:



Assemblies already store rich metadata (types, methods, parameters, attributes). Reflection is the API that **reads that metadata** and optionally **calls** what it finds.

2. Why reflection exists

Common real uses:

Scenario	How reflection helps
Frameworks (ASP.NET , EF, serializers)	Map JSON/HTTP/SQL to your types without generating code for every class by hand
Plugins / add-ins	Load a DLL and find types that implement <code>IPlugin</code>
DI / “scan and register”	Find all classes ending with <code>Service</code> and register them
Attributes	Read <code>[Authorize]</code> , <code>[HttpGet]</code> , custom markers and change behavior
Tools / diagnostics	Object dumpers, mappers, test helpers
Late binding	Call a method whose name comes from config

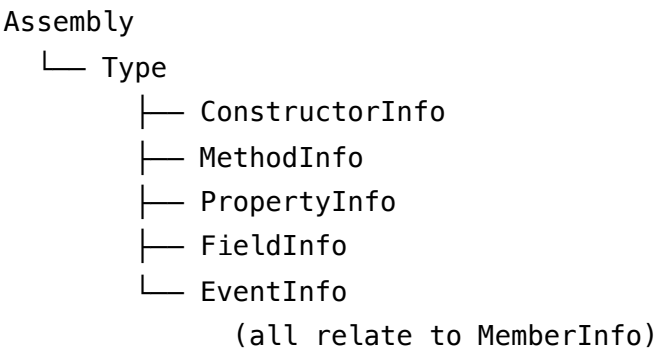
Without reflection, frameworks would need you to write huge amounts of glue code for every type.

3. Core types you will use

Type	Role
Assembly	A loaded DLL/EXE; entry point to its types
Type / TypeInfo	A type’s metadata (class, struct, interface, ...)
MemberInfo	Base for members (methods, fields, properties, ...)
MethodInfo	A method you can inspect or <code>Invoke</code>
ConstructorInfo	A constructor you can <code>Invoke</code>

Type	Role
PropertyInfo	A property; GetValue / SetValue
FieldInfo	A field; GetValue / SetValue
ParameterInfo	Parameter name, type, optional/default
EventInfo	Event metadata
CustomAttributeData / attribute APIs	Read attributes on types/members
Activator	Convenient CreateInstance
BindingFlags	Filter what Get* methods return (public/non-public, static/instance, ...)

Hierarchy (simplified):



4. Getting a Type

```
// 1) Compile-time known type
Type t1 = typeof(string);
Type t2 = typeof(List<>);           // open generic
Type t3 = typeof(List<int>);       // closed generic

// 2) From an instance
object value = "hello";
Type t4 = value.GetType();

// 3) From a name (runtime string) – must be assembly-qualified or findable
Type? t5 = Type.GetType("System.Int32");
Type? t6 = Type.GetType("Acme.Shop.Orders.Order, Acme.Shop");
```

`typeof` is preferred when you know the type at compile time (safer, faster lookup path).

`Type.GetType(string)` is for truly dynamic names — easy to get wrong if the assembly isn't loaded / name isn't qualified.

5. Inspecting assemblies and types

```
using System.Reflection;

Assembly asm = typeof(Program).Assembly;           // this app
// Assembly asm = Assembly.LoadFrom("Plugin.dll"); // another DLL

Console.WriteLine(asm.FullName);

foreach (Type type in asm.GetTypes())
{
    if (!type.IsClass || type.IsAbstract)
        continue;

    Console.WriteLine(type.FullName);
    Console.WriteLine($"  Public: {type.IsPublic}");
    Console.WriteLine($"  Nested: {type.IsNested}");
}
```

Useful Type flags:

Property	Meaning
IsClass / IsValueType / IsInterface / IsEnum	Kind of type
IsAbstract / IsSealed	Abstract / sealed
IsGenericType / IsGenericTypeDefinition	Generics
BaseType	Parent class
GetInterfaces()	Implemented interfaces

6. Members: properties, methods, fields

Sample type:

```

public class Product
{
    public string Name { get; set; } = "";
    public decimal Price { get; private set; }

    public void ApplyDiscount(decimal percent)
    {
        Price *= (1 - percent);
    }

    private void Touch() { }
}

```

Properties

```

Type type = typeof(Product);
object product = Activator.CreateInstance(type)!;

PropertyInfo name = type.GetProperty("Name");
name.SetValue(product, "Tea");
Console.WriteLine(name.GetValue(product)); // Tea

foreach (PropertyInfo prop in type.GetProperties())
{
    Console.WriteLine($"{prop.Name}: {prop.PropertyType.Name}");
}

```

Methods

```

MethodInfo? apply = type.GetMethod("ApplyDiscount");
apply!.Invoke(product, new object[] { 0.10m }); // 10% off

```

BindingFlags (public vs private, static vs instance)

By default, `GetMethod("Touch")` returns `null` for a **private** method. Include non-public:

```
MethodInfo? touch = type.GetMethod(
    "Touch",
    BindingFlags.Instance | BindingFlags.NonPublic);

touch?.Invoke(product, null);
```

Common flag combinations:

```
BindingFlags.Public | BindingFlags.Instance
BindingFlags.Public | BindingFlags.Static
BindingFlags.NonPublic | BindingFlags.Instance
BindingFlags.Public | BindingFlags.NonPublic | BindingFlags.Instance | BindingFlags.Sta
```

Reflection **can** call private members. That is powerful for frameworks/tests — and dangerous if abused (breaks encapsulation, brittle against renames).

7. Creating instances

```
// Parameterless constructor
object? a = Activator.CreateInstance(typeof(Product));

// Constructor with parameters
object? b = Activator.CreateInstance(
    typeof(List<int>),
    new object[] { /* capacity */ 16 });

// Via ConstructorInfo
ConstructorInfo ctor = typeof(Product).GetConstructor(Type.EmptyTypes)!;
object c = ctor.Invoke(null);
```

Generic types:

```
Type open = typeof(List<>);
Type closed = open.MakeGenericType(typeof(string));
object list = Activator.CreateInstance(closed)!; // List<string>
```

8. Practical example: simple property mapper

```
static void CopyPublicProperties(object source, object target)
{
    Type srcType = source.GetType();
    Type dstType = target.GetType();

    foreach (PropertyInfo srcProp in srcType.GetProperties())
    {
        if (!srcProp.CanRead)
            continue;

        PropertyInfo? dstProp = dstType.GetProperty(srcProp.Name);
        if (dstProp is null || !dstProp.CanWrite)
            continue;

        if (!dstProp.PropertyType.IsAssignableFrom(srcProp.PropertyType))
            continue;

        object? value = srcProp.GetValue(source);
        dstProp.SetValue(target, value);
    }
}

var dto = new { Name = "Tea", Price = 2.5m };
var entity = new Product();
CopyPublicProperties(dto, entity);
```

This is the spirit of many mappers/serializers (real libraries are more optimized and feature-rich).

9. Attributes and reflection

Attributes are metadata you attach in source; reflection reads them at runtime.


```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method)]
public sealed class RouteAttribute : Attribute
{
    public string Template { get; }
    public RouteAttribute(string template) => Template = template;
}
```

```
[Route("api/products")]
public class ProductsController
{
    [Route("")]
    public void List() { }

    [Route("{id}")]
    public void GetById(int id) { }
}
```

Read them:

```
Type type = typeof(ProductsController);
```

```
RouteAttribute? typeRoute = type.GetCustomAttribute<RouteAttribute>();
Console.WriteLine(typeRoute?.Template); // api/products
```

```
foreach (MethodInfo method in type.GetMethods(BindingFlags.Public | BindingFlags.Instance))
{
    RouteAttribute? route = method.GetCustomAttribute<RouteAttribute>();
    if (route is null) continue;

    Console.WriteLine($"{method.Name} → {route.Template}");
}
```

This pattern powers:

- [ASP.NET](#) routing / [HttpGet]
- validation [Required]
- ORMs / serializers
- your own markers ([Job] , [Command] , ...)

10. Practical example: discover plugins by interface

Same idea as assembly loading, but discovering types:

```
using System.Reflection;

public interface IPlugin
{
    string Name { get; }
    void Execute();
}

public static class PluginScanner
{
    public static IEnumerable<IPlugin> LoadPlugins(Assembly assembly)
    {
        foreach (Type type in assembly.GetTypes())
        {
            if (type.IsAbstract || type.IsInterface)
                continue;

            if (!typeof(IPlugin).IsAssignableFrom(type))
                continue;

            if (Activator.CreateInstance(type) is IPlugin plugin)
                yield return plugin;
        }
    }
}

// Usage
foreach (IPlugin plugin in PluginScanner.LoadPlugins(typeof(Program).Assembly))
{
    Console.WriteLine($"Running {plugin.Name}");
    plugin.Execute();
}
```

With a separate plugin DLL:

```

Assembly pluginAsm = Assembly.LoadFrom("plugins/SamplePlugin.dll");
foreach (IPlugin plugin in PluginScanner.LoadPlugins(pluginAsm))
{
    plugin.Execute();
}

```

(For version isolation across plugins, combine this with `AssemblyLoadContext` — see the assemblies guide.)

11. Practical example: scan and register services

```

public interface IOrderService { }

public class OrderService : IOrderService { }
public class PricingService { } // not registered – no matching interface rule

static void RegisterByConvention(Assembly assembly)
{
    foreach (Type impl in assembly.GetTypes())
    {
        if (!impl.IsClass || impl.IsAbstract)
            continue;

        // Convention: class OrderService → interface IOrderService
        Type? iface = impl.GetInterface("I" + impl.Name);
        if (iface is null)
            continue;

        Console.WriteLine($"Register {iface.Name} → {impl.Name}");
        // In real ASP.NET: services.AddScoped(iface, impl);
    }
}

```

Reflection finds the mapping; the DI container stores the result. Do the scan **once at startup**, not per request.

12. Performance — reflection is slower

Reflection costs more than direct calls because of metadata lookup, binder checks, and often boxing/ object parameters.

Rough guidance:

Approach	Relative idea
Direct call <code>product.Name</code>	Fastest
Cached <code>PropertyInfo + GetValue</code>	Much slower than direct, often OK at startup
<code>GetProperty</code> every time in a hot loop	Worst — avoid

Cache what you look up

```
using System.Reflection;

static class PropCache<T>
{
    private static readonly Dictionary<string, PropertyInfo?> Cache = new();

    public static PropertyInfo? Get(string name)
    {
        if (Cache.TryGetValue(name, out PropertyInfo? prop))
            return prop;

        prop = typeof(T).GetProperty(name);
        Cache[name] = prop;
        return prop;
    }
}

// Hot path: lookup is cached
var prop = PropCache<Product>.Get("Name");
var value = prop?.GetValue(product);
```

Rules of thumb:

1. Reflect **once at startup** (scan attributes, build dictionaries), then use normal calls.
2. Never `GetCustomAttribute` / `GetMethod` inside a per-request hot loop without caching.
3. For ultra-hot paths, prefer **direct calls** (normal C#) instead of reflection.
4. Prefer generic helpers (`GetCustomAttribute<T>()`) over weakly typed arrays when possible.

13. Safety and design caveats

Risk	Why it matters
No compile-time check	Typos in <code>"ApplyDiscount"</code> fail at runtime
Breaks encapsulation	Non-public access couples you to private details
Rename / trim / AOT	Linkers and Native AOT may remove members reflection expected
Security	Loading arbitrary DLLs + reflecting them is a trust boundary
Complexity	Harder to read/debug than normal OOP

Trimming / Native AOT note: if you publish trimmed or AOT, keep reflection-visible APIs with annotations (`[DynamicallyAccessedMembers]` , root descriptors), or reduce reflection usage.

14. Alternatives (when reflection is the wrong tool)

Need	Prefer instead
Known collaboration	Interfaces / generics
Serialization	<code>System.Text.Json</code>
Plugin contracts	Shared interfaces + ALC (not stringly method names)
Perf-critical member access	Direct code (normal property/method calls)

Reflection is a **tool for frameworks and extensibility**, not a default style for everyday business logic.

15. Quick API cookbook

```
typeof(User)           // Type
obj.GetType()          // Type from instance
Assembly.GetExecutingAssembly() // current assembly
asm.GetTypes()         // all types
type.GetProperties()    // public instance props
type.GetMethods(BindingFlags...) // filtered methods
type.GetCustomAttribute<T>() // attribute on type
method.GetCustomAttribute<T>() // attribute on method
Activator.CreateInstance(type) // new instance
method.Invoke(instance, args) // call method
prop.GetValue(instance) / SetValue(...) // read/write property
type.MakeGenericType(typeof(int)) // close open generic
typeof(IPlugin).IsAssignableFrom(type) // implements / inherits?
```

16. Key takeaway

Reflection reads assembly metadata at runtime so you can discover types, read attributes, create instances, and invoke members without hard-coding every name.

Use it for **plugins, frameworks, convention scanning, and attribute-driven behavior**.

Cache aggressively, keep it out of hot paths, respect trust boundaries when loading DLLs, and prefer interfaces / direct code when the problem doesn't truly need runtime discovery.